

Implementation of Clock Gating

Introduction

Clock gating is a power-saving technique used in digital designs to reduce dynamic power consumption. It allows selective enabling or disabling of the clock signal to various blocks of a chip. When a block does not need to function, clock gating disables its clock, preventing unnecessary circuit transitions (toggles) and saving power.

In this experiment, you will learn the behavior and power implications of a standard D flip-flop versus one with clock gating, using Verilator and GTKWave.

For a gated clocked flip-flop, the Boolean expression for the gated clock is:

$$\text{gated_clk} = \text{clk} \ \& \ \text{enable}$$

Where:

- **clk**: Main clock input
- **enable**: Clock gating control
- **gated_clk**: Clock delivered to the flip-flop

Block Diagram :

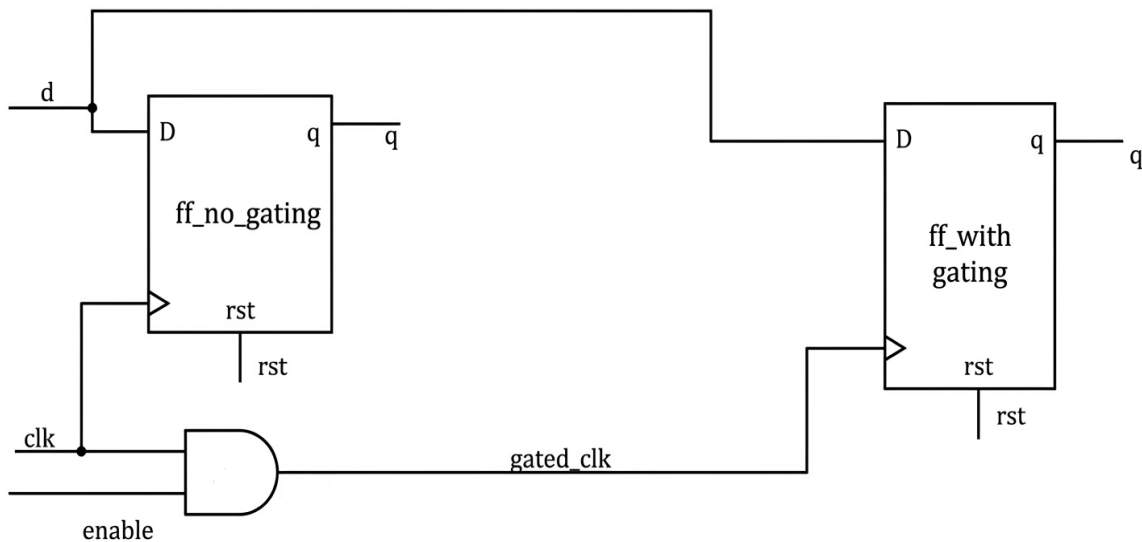


Fig : Block diagram of flip flop with clk and gated_clk

Applications:

Clock gating is used in:

- Microprocessors
- SoCs (System-on-Chip)
- Low-power embedded systems
- Power management units
- Any digital design requiring dynamic power reduction

By halting clock signals to idle blocks, dynamic power (the largest contributor to digital power consumption) is minimized, making clock gating vital in modern VLSI design.

Procedure To Create Project:

Step 1: Set up Project Directory

1.1: Create a directory for your lab:

```
mkdir clock_gating_lab
```

1.2: Open lab directory:

```
cd clock_gating_lab
```

Step 2: Create Design Files

2.1: Regular Flip-Flop without Gating (ff_no_gating.v)

```
module ff_no_gating (  
    input clk,  
    input rst,  
    input d,  
    output reg q  
);  
always @(posedge clk) begin  
    if (rst)  
        q <= 0;  
    else  
        q <= d;  
    end  
endmodule
```

2.2: Flip-Flop with Clock Gating (ff_with_gating.v)

```
module ff_with_gating (  
    input clk,  
    input rst,  
    input d,  
    input enable,  
    output reg q  
);  
    wire gated_clk;  
    assign gated_clk = clk & enable;  
    always @(posedge gated_clk) begin  
        if (rst)  
            q <= 0;  
        else  
            q <= d;  
    end  
endmodule
```

Step 3: Create Testbench File

3.1: Testbench (ff_tb.v)

```
module ff_tb;  
    reg clk = 0, rst = 0, d = 0, enable = 0;  
    wire q_nogate, q_gate;  
    always #5 clk = ~clk;  
    ff_no_gating    u1 (.clk(clk), .rst(rst), .d(d), .q(q_nogate));  
    ff_with_gating  u2 (.clk(clk), .enable(enable), .rst(rst),  
    .d(d), .q(q_gate));  
    initial begin  
        $dumpfile("dump.vcd");  
        $dumpvars(0, ff_tb);  
    end  
endmodule
```

```
rst = 1; d = 0; enable = 0; #10 rst = 0;
d = 1; #20;
enable = 1; #30;
enable = 0; d = 0; #30;
$finish;

end

endmodule
```

3.2: Save all files in the project directory.

Step 4: Simulation Script

4.1: Create “run.sh”

```
#!/bin/bash

verilator --binary -j 0 -Wall ff_no_gating.v \
ff_with_gating.v ff_tb.v --top ff_tb --timing \
--CFLAGS "-std=c++20" --trace
cd obj_dir || { echo "obj_dir not found"; exit 1; }
make -f Vff_tb.mk Vff_tb || { echo "Compilation failed"; exit
1; }

./Vff_tb || { echo "Simulation failed"; exit 1; }

gtkwave dump.vcd
```

Make the script executable:

```
chmod +x run.sh
```

4.2: Run the script to perform compilation, simulation, and open the waveform.

Simulation Output:

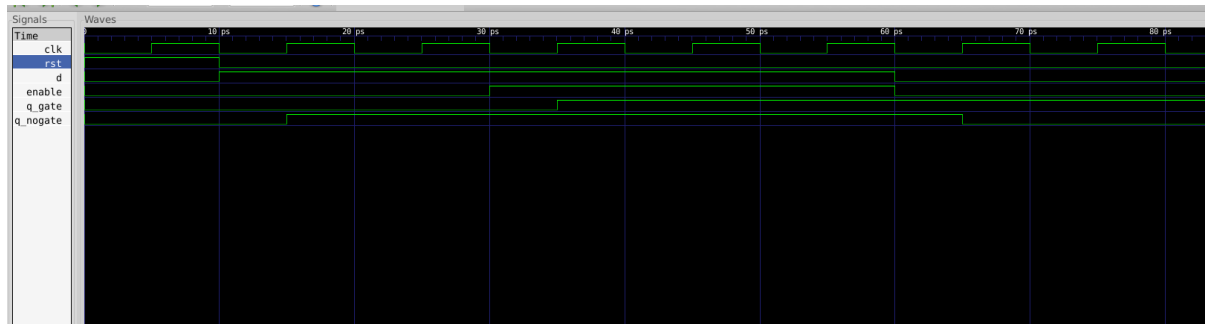
The simulation demonstrates:

- **clk** toggles continuously.
- **enable** controls clock gating.
- **q_nogate** toggles on every clock edge if data changes.
- **q_gate** only toggles when **enable** is high, reducing unnecessary switching and thus power usage.

Step 5: Viewing the Waveform

- Observe that **q_gate** only changes value when **enable** is high.
- When **enable** is low, the gated flip-flop output remains unchanged, confirming clock gating effectiveness.

Output Waveform:



- X-axis: simulation time
- Y-axis: level of each signal (0 = low, 1 = high)
- You will see fewer toggles on **q_gate** than on **q_nogate**, proving lower dynamic power usage in the gated case.

Conclusion:

Clock gating is a practical and widely used technique to reduce dynamic power consumption in digital integrated circuits. By selectively enabling the clock only when a circuit element is active, unnecessary transitions are prevented, which is directly visible through simulation waveforms and quantitative toggling behavior. This experiment helps to visualize, implement, and understand the impact of clock gating at the RTL design level using Verilator and GTKWave.